Production-Grade Roadmap: Decoupled Student Dashboard

> **Project:** Student Dashboard for Stephotec Computer Technologies Ltd.
> **Architecture:** Decoupled Full-Stack Engine (Backend-First Strategy)
> **Core Stack:** Django REST Framework (Backend) & Next.js (Frontend)
> **Target Hosting:** Render (Backend / PostgreSQL) & Vercel (Frontend)

This document establishes an industry-standard roadmap designed to implement a high-security, scalable, decoupled student ecosystem. Development is explicitly split into a backend-first phase to ensure fully documented, self-contained RESTful APIs via Swagger prior to frontend consumption.

Phase 1: Foundation & Architecture (Django)

In production environments, engineering begins with structural predictability and local-to-production environment matching.

- **Environment Isolation:** Initialize a virtual environment (`venv` or `poetry`), using deterministic dependency freezing for version consistency across local setups and the cloud.
- **Database Parity:** Utilize local `PostgreSQL` (ideally via Docker) rather than SQLite to maintain exact environment alignment with the target hosting on Render.
- **Custom User Model:** Extend Django's `AbstractUser` immediately. Swapping user models mid-project creates complex migration issues. This enables the use of `student_id` as the primary authentication field alongside custom roles (Admin vs. Student).

Phase 2: Core Backend Development

Building the internal data engine and automated provisioning layers for Stephotec Computer Technologies Ltd.

1. Data Modeling & File Routing

- **Courses:** A `Course` model containing unique indicators, descriptive naming, schema codes, and duration parameters.
- **Profiles:** A separate `StudentProfile` tied via a strict `OneToOneField` to the Custom User model to hold bio text, physical addresses, and external cloud assets (configured with Cloudinary or AWS S3 for production file uploads).

2. Collison-Free ID Generation Logic

- Implement automated, rule-based database hooks (e.g., overriding the `save()` method or signal handling). When an admin assigns a student to "Software Engineering" (SE) in 2026, the backend calculates the next incremental sequence to output structured identifiers such as `ST-SE-2026-001`. This identifier functions natively as their unique login username.

3. Authentication & State Flags

- **Token Security:** Implement stateless JSON Web Tokens via `django-rest-framework-simplejwt`.
- **First-Time Activation Flow:** Define boolean markers like `is_profile_complete`. Upon initial login using the admin-generated temporary password, the system enforces a strict password update and profile completion workflow.

Phase 3: API Refinement & Open Documentation

Isolating application logic into decoupled REST interfaces designed for clean consumption by any external client framework.

- **Serializers & ViewSets:** Construct isolated data translation layers via DRF to manage clean validation schemas.
- **Granular Permission Matrices:** Secure endpoints with DRF standard policies (`IsAuthenticated`) combined with custom class layers (e.g., `IsAdminUser` for student account setup, and `IsOwner` for secure profile updates).
- **Interactive Documentation:** Embed `drf-spectacular` to automatically export a standard OpenAPI 3 schema, serving interactive, real-time testing frameworks via Swagger UI at `/api/schema/swagger-ui/`.

Phase 4: Production Readiness & Backend Hosting

Transitioning the Django architecture into a production cloud environment prior to writing frontend code.

- **Environment Secrets:** Implement `django-environ` to completely abstract operational data (database credentials, private encryption keys, CORS patterns) directly from the codebase into isolated system environments.
- **Enterprise Servers:** Configure high-concurrency WSGI/ASGI engines like `gunicorn` or `uvicorn` for production traffic management.
- **Render Cloud Deployment:** Setup automatic build blueprints via `render.yaml` linked to your GitHub repository, orchestrating automatic continuous integration, database migration passes, and static assets compilation.

Phase 5: Frontend Development (Next.js)

With the backend APIs live, validated, and interactive via Swagger, development shifts to building the client interfaces.

- **Framework Adoption:** Transition your React foundations into Next.js by adopting the modern App Router architecture, file-based routing mechanisms, Server Actions, and optimized client/server fetching strategies.
- **Secure Session Management:** Construct state contexts or secure HTTP-only cookies to handle JWT lifetimes, backed by Next.js Middlewares to guard private dashboard views against unauthorized access.
- **Professional Interface Design:** Build user-friendly dashboard panels utilizing Tailwind CSS complemented by high-grade component structures such as `shadcn/ui`.

Phase 6: Vercel Deployment & Cross-Origin Configuration

- Deploy the Next.js frontend directly to **Vercel** with full continuous delivery pipelines.
- Expose the live Render API URL to the client using structured frontend environment files (`NEXT_PUBLIC_API_URL`).
- Lock down the backend architecture using `django-cors-headers` to authorize API handshakes exclusively from your designated Vercel production domain.

Next Step Suggestion: When you are ready to begin, we can write the Custom User Model alongside the automated, course-based Student ID generation logic in Django.